

Writeup: du crackme "int_80"

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_05_int_80OK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `K3Y5-r3cK-H4xX0r`

Résumé

Introduction

Ce write-up présente la solution d'un reverse engineering d'un binaire ELF 64-bit nommé "int_80" l'objectif était de trouver un flag de 16 caractères.

Analyse initiale

Exécution de file qui confirme un exécutable ELF 64-bit statiquement lié

Extraction des chaînes de caractères avec strings révélant des indices incluant des messages comme "Entrez le flag" et plusieurs messages de distraction

Outils Utilisés

Extraction des chaînes de caractères avec strings révélant des indices incluant des messages comme "Entrez le flag" et plusieurs messages de distraction

Analyse Statique

Analyse approfondie

Examen des sections et symboles

Utilisation de readelf et objdump pour analyser la structure du binaire :

Identification des sections importantes (.text, .rodata, .data, .bss)

La table des symboles révèle des noms intéressants comme flag_parts, flag_indices, et construct_flag

Désassemblage et dynamique du programme

Le désassemblage avec objdump -d et radare2 montre que le programme :

Demande un flag à l'utilisateur

Analyse Dynamique

Exécution de file qui confirme un exécutable ELF 64-bit statiquement lié

Identification du Mécanisme de Validation

Reconstruit le flag correct à partir de deux tableaux en mémoire

Découverte du Flag

Reconstruit le flag correct à partir de deux tableaux en mémoire

Compare l'entrée utilisateur avec le flag reconstruit

Extraction du flag

L'analyse de la fonction `construct_flag` (à l'adresse `0x401000`) révèle :

Un tableau `flag_parts` à l'adresse `0x404607` contenant des caractères séparés par des `0xFF`

Un tableau `flag_indices` à l'adresse `0x404707` contenant des indices

Un algorithme qui utilise ces indices pour extraire les bons caractères de `flag_parts`

En examinant les valeurs des tableaux avec `pcs` et `pxw` dans `radare2` :

Les caractères significatifs dans `flag_parts` sont : K, 3, Y, 5, -, r, 3, c, K, -, H, 4, x, X, 0, r

Les indices dans `flag_indices` permettent de reconstruire le flag dans le bon ordre

Solution

Le flag correct est : K3Y5-r3cK-H4xX0r

Cette solution confirme les indices trouvés dans le binaire, notamment l'une des chaînes `"fake_flag"` qui ressemblait à cette syntaxe.

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.